

Field	Details
Institute / University	Pimpri Chinchwad College of Engineering ,Pune
Programme / Batch	B.Tech Computer Engineering / Expected graduation: 2027
Student Name	Aditi Ashok Bachulkar
PRN	123B1B085
Case Study ID (CS1–CS5)	CS1 – AUTOSAR HLD Document Analysis Assistant
Project Title	AUTOSAR HLD Document Analysis Assistant using RAG

1. Problem Statement and Intended Users

1.1 The problem

An AUTOSAR High-Level Design (HLD) document explains how a vehicle software system is planned at a high level. These documents are usually long. They contain many kinds of information, such as:

- software components and what each one does
- interfaces and ports
- signals and data flows
- communication details (for example CAN)
- dependencies between components
- diagnostics
- the overall system architecture

When an engineer needs one small fact, for example “which signals does this component receive?”, they have to scroll through many pages or use plain keyword search. Keyword search does not help much when the document uses different words for the same idea. Searching manually takes time and it is easy to miss a related section on another page.

1.2 Proposed solution

This project is an AI-assisted prototype that follows the Retrieval-Augmented Generation (RAG) idea. The user uploads an HLD PDF, builds a searchable knowledge base from it, and then asks questions in normal English. The system finds the parts of the document that are most related to the question and shows an answer built from those parts. In the demonstrated Demo Extractive Mode, the answer is taken from the retrieved text and no API key is needed. Every answer comes with source page and section citations, so the user can open the PDF and check the source.

Important: this is an AI-assisted document analysis tool for learning and demonstration. It does not replace an automotive engineer, and it is not an official OEM engineering tool. A synthetic (made-up) HLD document is used because real OEM AUTOSAR documents are proprietary.

1.3 Intended users

- Automotive software engineers and developers who need to look up information in an HLD
- Testers who need to find and understand design information before testing
- Software architects who want to check how components connect
- Technical reviewers who want a fast first pass over a document
- Automotive engineering students who are learning how HLD documents are structured

2. Input Data / Knowledge Base

The knowledge base is one synthetic AUTOSAR HLD PDF that is uploaded by the user. The table below gives a short summary. More details follow after the table.

Item	Description
Sources	Synthetic / educational AUTOSAR HLD document prepared for demonstration. No proprietary OEM document was used. File name: sample_hld.pdf How it was prepared: the sample HLD PDF was generated as part of the project for testing and demonstration.
Size and format	Format: PDF with selectable text. Number of pages: 11 File size: Not recorded in the verified project configuration. Number of chunks created: 11 (pages processed: 11, indexed documents: 11)
Preprocessing EDA done	The PDF is processed page by page: the text is extracted, divided into chunks and stored in a ChromaDB knowledge base (details in Section 2.2). For the sample HLD, 11 pages were processed, 11 chunks were created and 11 documents were indexed.
Chunking strategy	Chunk size: 850 characters (character-based, not token-based). Overlap: 150 characters. For the sample HLD, 11 chunks were created from 11 pages. Each chunk keeps its page number and section so that the citation can be shown.

2.1 About the demonstration document

The HLD used here is synthetic and educational. It was made only for demonstration and evaluation, so it does not describe a real vehicle or a confidential OEM design. The document contains technical information about the following topics:

- Architecture overview
- Vehicle Application Manager
- Communication Architecture
- Communication Manager
- CAN Interface
- Sensor and Actuator Services
- Diagnostic information
- CAN signals and message identifiers

For example, the CAN signals DoorLockRequest, CabinSetpoint and DoorStateStatus appear in the answer to the demonstrated question (see Section 5).

2.2 Preprocessing steps

1. The user uploads the HLD PDF in the Streamlit app.
2. Text is extracted from the PDF.
3. The page number of every piece of text is kept.
4. The extracted text is processed before chunking. The text of each page is stripped of leading and trailing spaces, and pages with no text are ignored. No other cleaning (such as OCR, header and footer removal, duplicate removal or lowercasing) is done.
5. Section headings are identified where possible. The section is taken from the first line, among the first 12 lines of the page, that starts with a digit, is in uppercase or starts with “#”. If no such line is found, the section is “General”.
6. The text is divided into chunks.
7. Metadata is attached to every chunk.
8. An embedding (a list of numbers that represents the meaning of the text) is generated for every chunk.
9. The chunks, embeddings and metadata are stored in ChromaDB.

The prototype works with the text extracted from the PDF. Text that exists only inside images or scanned pages may not be retrieved correctly.

2.3 Chunk metadata

Each chunk is stored together with the following information. This is what makes citations possible.

Metadata field	Meaning
Source document	Name of the uploaded PDF the chunk came from.
Page number	Page of the PDF on which the chunk text was found.
Section	Section heading of the chunk, for example “5. Communication Architecture”.
Chunk ID	A unique ID for the chunk, used to store and find it in ChromaDB.

3. Solution Architecture

Component	Choice (name, version, hosting)
Pre-trained model (LLM)	Demonstrated response mode: Demo Extractive Mode. The answer is built from the retrieved HLD text and no API key is needed. LLM provider and model (only if an LLM mode is available in the code): an optional OpenAI-compatible Chat Completions API branch exists in the code (provider “openai”, set with the environment variables LLM_PROVIDER, LLM_API_KEY and LLM_MODEL). It was not used in the demonstrated evaluation, and the model name is not fixed in the code.

Component	Choice (name, version, hosting)
Embedding model	Embedding model: all-MiniLM-L6-v2, loaded with the sentence-transformers library (version 6.1.0). The embeddings are normalized. It creates embeddings for every chunk and for every question.
Vector store (FAISS / ChromaDB)	ChromaDB, used as the vector knowledge base. Version: 1.5.9. Storage folder: vector_db/ (persistent local storage; collection name hld_chunks; cosine distance).
RAG pipeline / AI-agent design	A retrieval pipeline: the question is converted to an embedding, the relevant chunks are retrieved from ChromaDB, and an extractive response with source page and section citations is built from them. No AI agents are used.
Service layer and UI	Python 3.14.6 application with a Streamlit web interface, run locally. Streamlit version: 1.65.0.
Tools / deterministic rules	Extractive response built only from the retrieved chunks. Source page and section citations and the evidence text are shown with every answer. Evaluation option with predefined test questions that calculates retrieval success rate, citation coverage, answer coverage and an overall score. Handling of questions that are not answered in the HLD: the message “I could not find sufficient evidence for this answer in the uploaded HLD.” is returned only when retrieval returns no results. There is no similarity threshold, so if the knowledge base is not empty, an unsupported question may still retrieve the closest chunks and the demo mode may return them with citations.

3.1 Architecture in simple steps

The system works in two main parts. First, the HLD PDF is processed and a knowledge base is built. Second, each question is answered from the retrieved content. A separate evaluation part runs the test questions and calculates the scores.

User → Streamlit User Interface → PDF Processing → Text Extraction → Document Chunking → Embedding Generation → ChromaDB Knowledge Base → Relevant Chunk Retrieval → Question + Retrieved Context → Extractive Response → Citation / Evidence → User

Evaluation: Test Questions → Retrieval Evaluation → Citation Evaluation → Answer Coverage Evaluation → Overall Score

AUTOSAR HLD Document Analysis Assistant - System Architecture

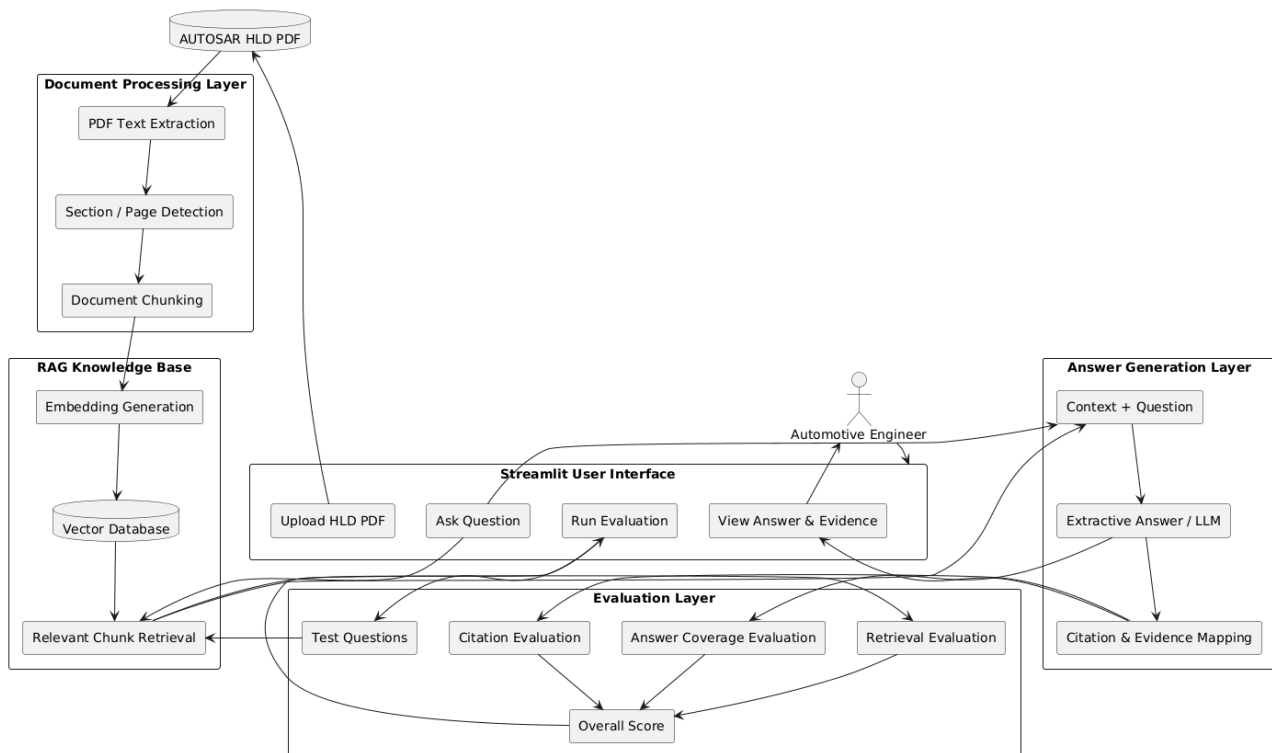


Diagram 1: System architecture of the AUTOSAR HLD Document Analysis Assistant

3.2 Components explained

- **Streamlit UI:** The web page the user works with. It has the PDF upload, the question box and the area where the answer and citations are shown.
- **PDF text extraction:** Reads the PDF and returns the text of each page. The page number is kept with the text.
- **Text processing and chunking:** Divides the extracted text into smaller pieces (chunks). Each chunk keeps its page number and section.
- **Embedding model:** Turns each chunk and each question into a vector of numbers. Texts with similar meaning get vectors that are close to each other.
- **ChromaDB:** Stores the chunk text, vectors and metadata as the knowledge base, and finds the chunks closest to a question vector.
- **Retriever:** Converts the question to an embedding and asks ChromaDB for the most relevant chunks.
- **Extractive response:** Uses the question and the retrieved context to build the answer from the retrieved text. In Demo Extractive Mode no API key is needed.
- **Citation and evidence display:** Shows the source file, page number and section of the chunks that were used, together with the evidence text, so the user can check the answer in the PDF.
- **Evaluation:** Runs the predefined test questions and calculates the retrieval success rate, citation coverage, answer coverage and overall score.

3.3 RAG pipeline

1. **Document ingestion** – the user uploads the HLD PDF.
2. **Text extraction** – the text of each page is extracted from the PDF.
3. **Chunking** – the extracted text is divided into chunks and metadata (page, section) is attached.

4. **Embedding generation** – each chunk is converted into a vector by the embedding model.
5. **Vector storage** – chunks, vectors and metadata are saved in ChromaDB.
6. **Question processing** – the user types a question and it is converted into a vector with the same model.
7. **Similarity retrieval** – ChromaDB returns the chunks closest to the question vector.
8. **Context creation** – the retrieved chunks are collected as the context for the question, with their page/section labels.
9. **Response construction** – the question and the retrieved context are used to build the response.
10. **Extractive response** – in Demo Extractive Mode the answer is built from the retrieved text. No API key is needed.
11. **Citation display** – the answer is shown together with the page/section of the sources used.

3.4 Answering only from the document

The assistant is designed to give answers based on the retrieved document content, and every answer is shown with source page and section citations. In the demonstrated Demo Extractive Mode, the answer is taken from the retrieved HLD text, so no outside knowledge is added.

Because the answer is built from retrieved text and shown with citations, the user can check it in the original PDF. This reduces the chance of made-up information compared with a system that answers from memory. It is not a perfect guarantee: if the wrong chunk is retrieved or the text is incomplete, the answer can still be incomplete or misleading, so the citations should always be checked.

What the system replies when the answer is not present in the HLD (for example a “not enough evidence” message): “I could not find sufficient evidence for this answer in the uploaded HLD.” This message is returned only when retrieval returns no results. There is no similarity threshold and no check of whether the retrieved text really answers the question. If the knowledge base is not empty, an unsupported question may therefore still retrieve the closest chunks, and the demo mode may return them with citations. The current system cannot detect every unsupported question.

4. Implementation and Configuration

4.1 Project files

The code is kept in the Code/ folder. The purpose of each file is given below in simple words.

File	Purpose
app.py	Main Streamlit application. Builds the knowledge base, shows the question box, the answer and the citations, and runs the evaluation.
config.py	Keeps the settings in one place (paths, chunk size, overlap, top-k, model names and environment variables).
create_sample_pdf.py	Generates the synthetic sample HLD PDF. It is not called by the Streamlit application during normal use.
ingestion.py	Opens the PDF, extracts the text of each page and creates the page records with page number and section.
embeddings.py	Loads the SentenceTransformer embedding model and creates embeddings for chunks and questions.

File	Purpose
vector_store.py	Creates and uses the ChromaDB collection: adds chunks and searches for similar chunks.
rag_pipeline.py	Embeds the question, retrieves the relevant chunks, and builds the answer with citations.
llm_provider.py	Contains the optional OpenAI-compatible API call and the default demo extractive response.
prompts.py	Defines the runtime system prompt and builds the question and context prompt.
evaluation.py	Runs the predefined test questions and saves the evaluation CSV and summary.
utils.py	Performs the page-aware, character-based chunking.

4.2 Environment and dependencies

The project is written in Python and was run successfully with Python 3.14.6 inside a virtual environment (.venv). The main parts are the Streamlit user interface and the ChromaDB vector knowledge base. Library used for PDF text extraction: PyMuPDF 1.28.2 (imported as fitz). Library used for embeddings: sentence-transformers 6.1.0 (model all-MiniLM-L6-v2). Exact library versions: Python 3.14.6, Streamlit 1.65.0, PyMuPDF 1.28.2, ChromaDB 1.5.9, sentence-transformers 6.1.0, torch 2.14.1, transformers 5.19.0, pandas 3.0.6 and matplotlib 3.11.2. The demonstrated Demo Extractive Mode does not need an API key. The file requirements.txt lists only minimum versions (streamlit>=1.32, PyMuPDF>=1.24, sentence-transformers>=2.6, chromadb>=0.5, pandas>=2.0 and matplotlib>=3.7). matplotlib is listed there but is not directly used by the current application code. No separate LLM library is needed, because the optional API call uses the standard urllib.request module of Python.

4.3 Configuration values

Setting	Value
Embedding model	all-MiniLM-L6-v2 (sentence-transformers 6.1.0, embeddings normalized)
Vector database	ChromaDB
Vector database folder / collection name	Folder: vector_db/ (persistent local ChromaDB). Collection name: hld_chunks
Chunk size	850 characters (character-based, not token-based)
Chunk overlap	150 characters
Top-k retrieval	Default 4 (the user interface allows 3 to 5)
Similarity measure / score limit	Cosine similarity (collection setting hnsw:space = cosine). No explicit similarity threshold, no reranking and no metadata filter are used.

Setting	Value
LLM provider	Not used in the demonstrated Demo Extractive Mode (the default provider is demo). An optional OpenAI-compatible API branch exists (provider “openai”, set with LLM_PROVIDER) but was not used in the verified evaluation.
LLM model name	Not set by default (empty). It is read from the LLM_MODEL environment variable if the optional API mode is used. No model name was verified.
Temperature (only if set in the code)	0 in the optional API mode. Not applicable to the Demo Extractive Mode.
API key needed? How is it stored?	No API key is needed for the demonstrated Demo Extractive Mode. In the optional API mode the key is read from the environment variable LLM_API_KEY and is not hard-coded.
Prompt configuration	The run-time prompt is defined in Code/prompts.py. The system prompt tells the assistant to use only the supplied context and not to invent facts, and build_prompt(question, context) builds the question and context prompt. The files Model_Prompts_Config\system_prompt.txt and rag_prompt.txt are supporting files that the application does not read at run time.
Command to run the app	Environment creation: python -m venv .venv Activation: .venv\Scripts\activate Run command: streamlit run Code\app.py

4.4 Workflow

The workflow below shows what happens from the moment the user starts the app until the answer is displayed.

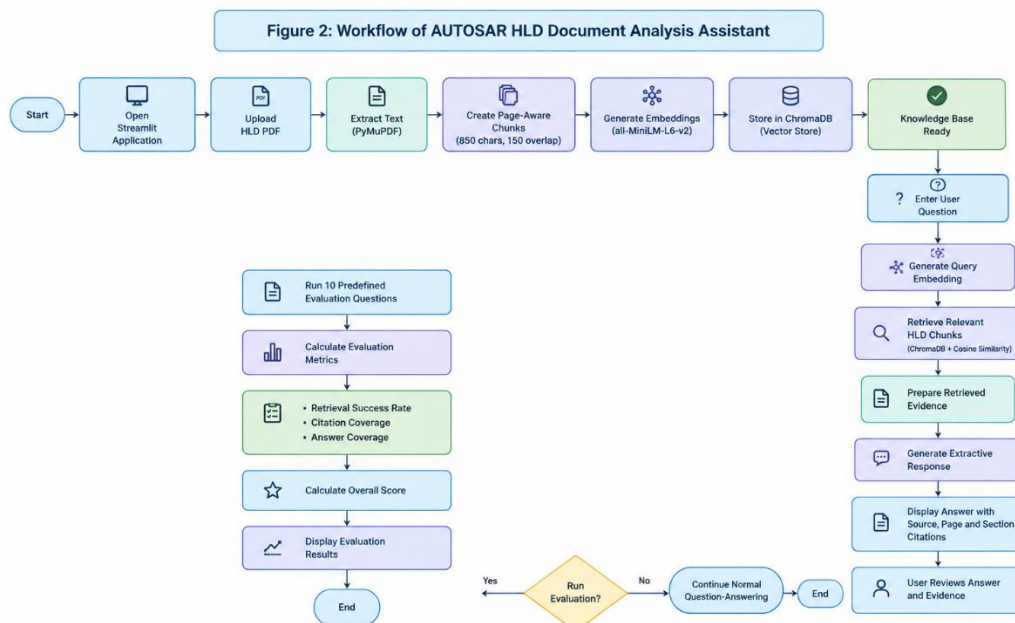


Diagram 2: Workflow of the AUTOSAR HLD Document Analysis Assistant

5. Evaluation Steps and Results

5.1 How the evaluation was done

1. A set of 10 predefined test questions about the synthetic HLD was used. Evaluation question file: Evaluation_Results\evaluation_questions.csv (the questions are run by Code\evaluation.py).
2. The evaluation can be started from the application after the knowledge base has been built.
3. The evaluation calculates the retrieval success rate, citation coverage, answer coverage and an overall score, and displays them in the application.
4. The results shown by the application were recorded in this report (Section 5.3).
5. The results apply only to the current synthetic HLD and evaluation setup and should not be generalized to real automotive HLD documents.

5.2 Test question set

Test No.	Question Scenario	Expected Answer / Behaviour	Actual Result	Pass / Fail	Notes
1	What is the purpose of the Communication Manager?	Coordinates application communication and CAN interface routing. Source page and section citation expected.	Explained that the Communication Manager coordinates communication between the application components and the CAN Interface.	Partial	Demonstrated question. Also mentioned DoorLockRequest, CabinSetpoint and DoorStateStatus. Source: sample_hld.pdf, Page 6, Section 5. Communication Architecture. Retrieval and citation: True. Answer coverage: 0.80.
2	Which component controls door lock and window functions?	Door Control Component controls door lock and window functions.	Answer built from the retrieved evidence, with a source citation (demo mode).	Pass	Retrieval and citation: True. Answer coverage: 1.00.
3	What signals does Sensor Manager acquire?	DoorPosition, CabinTemperature and OutsideTemperature.	Answer built from the retrieved evidence, with a source citation (demo mode).	Pass	Retrieval and citation: True. Answer coverage: 1.00.

Test No.	Question Scenario	Expected Answer / Behaviour	Actual Result	Pass / Fail	Notes
4	What is CAN ID for DoorLockRequest?	DoorLockRequest uses CAN ID 0x221.	Answer built from the retrieved evidence, with a source citation (demo mode).	Fail	Retrieval and citation: True. Answer coverage: 0.00. None of the expected keywords were found in the extractive answer, although a relevant page was retrieved.
5	What happens when a door actuator fault is reported?	Door lock request is rejected.	Answer built from the retrieved evidence, with a source citation (demo mode).	Pass	Retrieval and citation: True. Answer coverage: 1.00.
6	Which component owns controller access?	CAN Interface owns controller access.	Answer built from the retrieved evidence, with a source citation (demo mode).	Partial	Retrieval and citation: True. Answer coverage: 0.33. Only part of the expected keywords were found in the answer.
7	What does DTC-CLIM-014 mean?	Cabin-temperature sensor plausibility failure.	Answer built from the retrieved evidence, with a source citation (demo mode).	Pass	Retrieval and citation: True. Answer coverage: 1.00.
8	Describe the door lock functional flow.	Request passes CAN Interface, Communication Manager, Door Control and Actuator Manager.	Answer built from the retrieved evidence, with a source citation (demo mode).	Partial	Retrieval and citation: True. Answer coverage: 0.88. Only part of the expected keywords were found in the answer.

Test No.	Question Scenario	Expected Answer / Behaviour	Actual Result	Pass / Fail	Notes
9	What does Vehicle Application Manager depend on?	Communication Manager and Diagnostic Manager.	Answer built from the retrieved evidence, with a source citation (demo mode).	Pass	Retrieval and citation: True. Answer coverage: 1.00.
10	What is outside the demonstration scope?	Safety, cybersecurity, timing calibration and detailed AUTOSAR configuration.	Answer built from the retrieved evidence, with a source citation (demo mode).	Partial	Retrieval and citation: True. Answer coverage: 0.14. Only part of the expected keywords were found in the answer.

5.3 Metrics

The application reports the following scores for the 10 test questions. These are the actual results obtained from the prototype for the current synthetic HLD.

Metric	How it is measured	Score / observation
Retrieval success rate	Checks, for each of the 10 test questions, whether relevant HLD content was retrieved. A question counts as successful when at least one retrieved page is the same as at least one expected page. Retrieval success rate = $\text{mean}(\text{retrieval_success}) \times 100$.	100.0%
Citation coverage	Checks whether the answers include source page and section citations. A citation counts as present when the answer contains “[Source:”. Citation coverage = $\text{mean}(\text{citation_present}) \times 100$.	100.0%
Answer coverage	Checks how much of the expected answer content is covered by the answer. Only words longer than 4 characters in the expected answer are used as keywords, and the score is based on how many of these keywords are found in the answer. This is keyword-presence coverage. It does not measure semantic correctness or factual accuracy, and it was not checked by a human or expert.	71.5%

Metric	How it is measured	Score / observation
Overall score	Combined score calculated by the evaluation from the other scores. Overall Score = (Retrieval Success Rate + Citation Coverage + Answer Coverage) / 3.	90.5% (the average of 100.0%, 100.0% and 71.5% is 90.5%)
Latency	Time from asking a question to showing the answer.	Not implemented / not measured

These results apply to the current synthetic HLD and evaluation setup only. They do not show how the system would perform on real automotive HLD documents.

5.4 Failure cases and limitations

The following failure cases and limitations were identified for the current prototype:

- **Diagrams, images and complex tables:** information that is available only inside these may not be retrieved correctly, because the prototype mainly uses text extracted from the PDF.
- **Extractive answers:** the answer is made from retrieved text, so it may read less naturally than a fully generative LLM-based answer.
- **Partial answer coverage:** answer coverage was 71.5%, which is lower than retrieval success and citation coverage. This means the answers did not always cover everything that was expected.
- **Limited evaluation:** only 10 questions and one synthetic HLD were used, so the results should not be generalized to real automotive HLD documents.
- **Questions not answered in the HLD:** This case was not tested, because all 10 evaluation questions have answers in the HLD. In the code, the message “I could not find sufficient evidence for this answer in the uploaded HLD.” is returned only when retrieval returns no results. Since there is no similarity threshold, an unsupported question may still retrieve the closest chunks, and the demo mode may return them with citations.

Questions with weak or incomplete answers in the evaluation: question 4 (CAN ID for DoorLockRequest) had an answer coverage of 0.00, question 10 (outside the demonstration scope) had 0.14 and question 6 (controller access) had 0.33. Questions 1 (0.80) and 8 (0.88) were also only partly covered. In all of these cases a relevant page was retrieved and a citation was shown, but only some of the expected keywords were found in the extractive answer. Answer coverage is a keyword check, so a low value does not always mean that the answer is wrong, and the answers should still be checked against the cited pages.

6. Responsible AI Measures

Area	Measure adopted in this project
Grounding and citations	Answers are built from the retrieved HLD content only. The source page and section of the content are shown with the answer, so the user can verify the information in the original PDF.

Area	Measure adopted in this project
Privacy	A synthetic educational HLD is used instead of any confidential OEM document. The application was run locally on the student's computer. For real automotive documents, confidentiality and protection of proprietary information would be important, together with document access control, secure storage and version control.
Security	Security measures implemented in this prototype: the API key for the optional API mode is read from the environment variable LLM_API_KEY and is not hard-coded; the PDF file extension and existence are checked; errors are handled with friendly messages; and the retrieved evidence and citations are displayed for transparency. Authentication, role-based access control, encryption, audit logging, comprehensive input sanitization, document access control, secure storage and version control are not implemented in the current prototype.
Human oversight / review	The assistant only helps with searching and understanding the document. The engineer or user remains responsible for the final interpretation and must check the answer against the cited pages before using it in any decision.

Limitations: the answer can be incomplete or misleading if the retrieved context is incomplete or if the PDF text was extracted badly. The demonstrated extractive mode may also read less naturally than a generative answer. For this reason, the output should always be treated as a helpful first answer that needs to be reviewed, not as a final one.

Further limitations: The demonstrated mode is extractive: it returns the first retrieved evidence chunk (up to 700 characters), so it does not write a new answer. There is no similarity threshold and no reranking, so an unsupported question can still return the closest chunks with citations. The insufficient-evidence message appears only when nothing is retrieved. Answer coverage is a keyword check and does not measure semantic correctness. Latency was not measured. The optional LLM branch was not part of the verified evaluation, and the HLD used is synthetic and educational.

7. Innovation Highlights

This is a student capstone project, so the improvements below are practical and realistic, not a new research idea.

- **Natural-language querying of HLD documents:** the user can ask questions in normal English instead of searching by keywords.
- **Retrieval before answering:** the answer is built from the retrieved document content and not from memory.
- **Page/section citations:** each answer shows where the information came from, so it can be checked.
- **Vector knowledge base:** the HLD is stored as a searchable ChromaDB knowledge base that can be built or rebuilt from the application.
- **Simple Streamlit interface:** upload, ask, and view citations and evidence from one web page.
- **Evidence display:** the retrieved text is shown together with the answer, so the user can check where it came from.

- **Built-in evaluation:** predefined test questions can be run from the application and the scores are displayed.
- **Page and section aware metadata:** every chunk keeps its source file, page number and section, which makes the citations possible.
- **Transparent extractive demo mode:** the answer is taken directly from the retrieved HLD text and no API key is needed.
- **Optional LLM extension:** an optional OpenAI-compatible API branch exists in the code, but it was not used in the demonstrated evaluation.
- **Synthetic HLD for reproducible demonstration:** a synthetic HLD is used so that the demonstration and the evaluation can be repeated without any confidential document.

8. Screenshots, Logs and Evidence

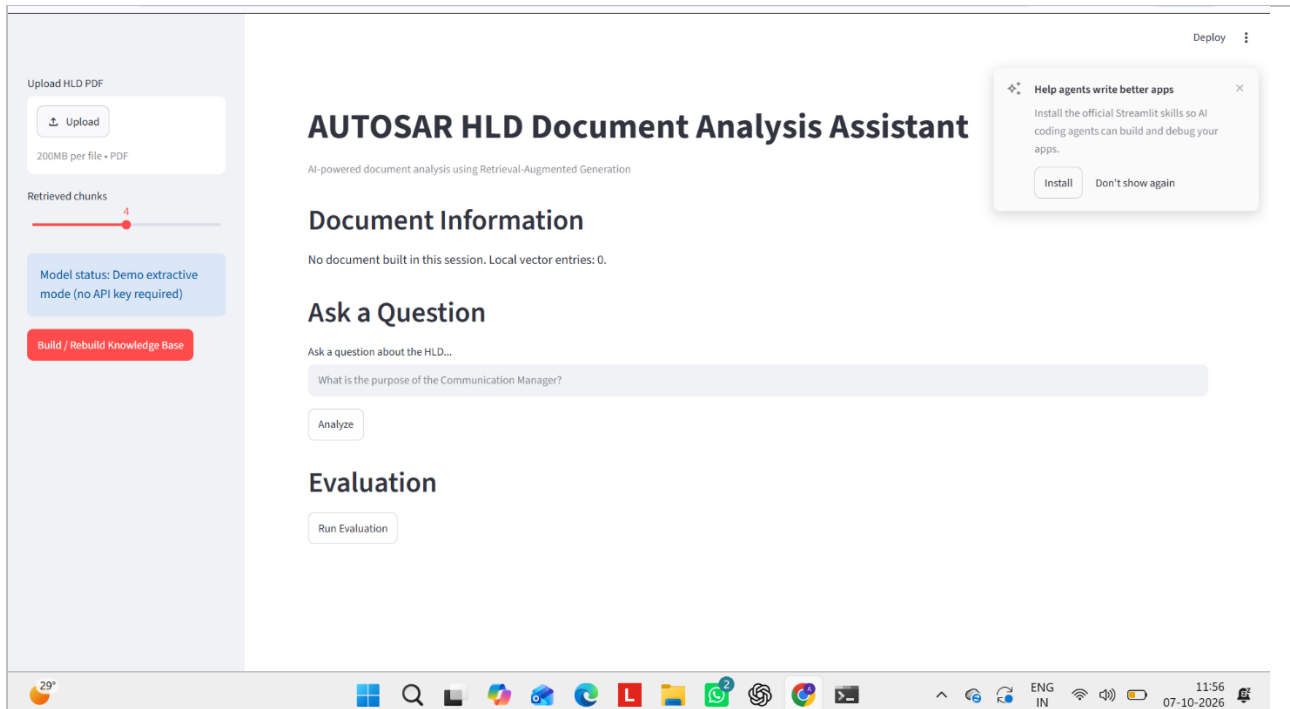


Figure 1: Main Streamlit application, showing that the application starts and what the interface lookslike.

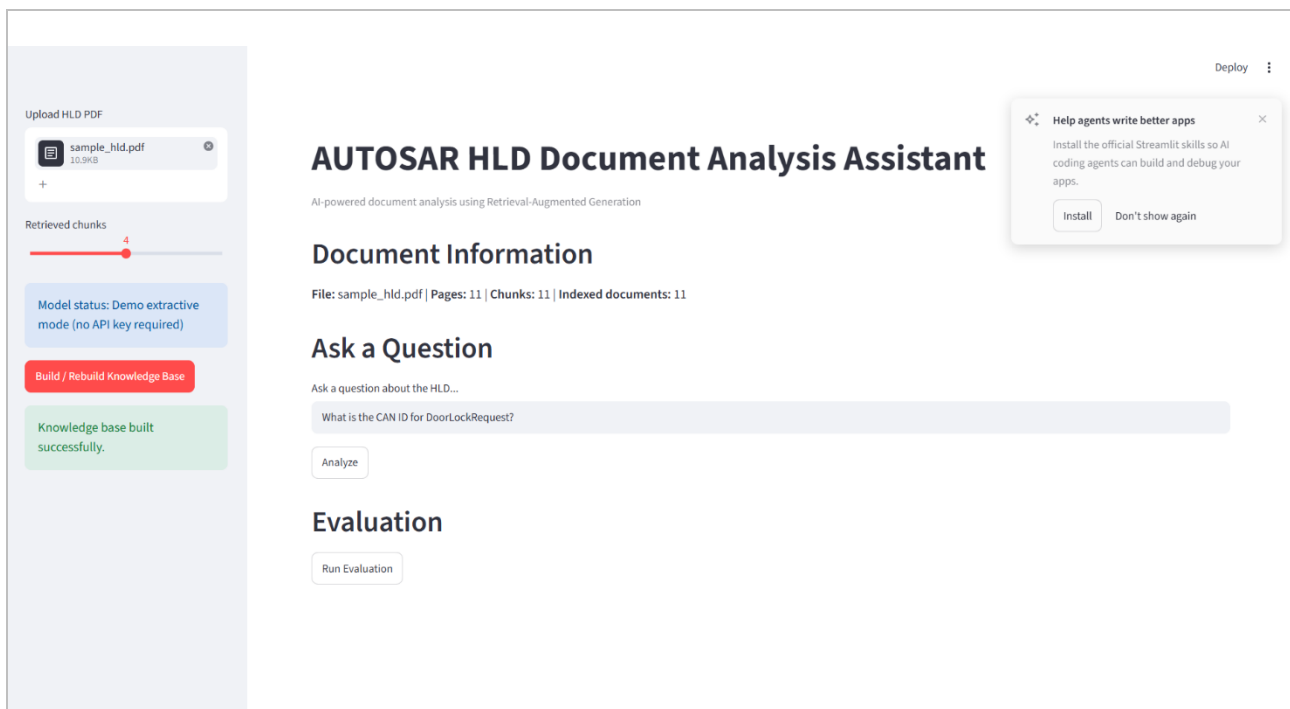


Figure 2: sample_hld.pdf uploaded in the application, showing the input document provided by the user.

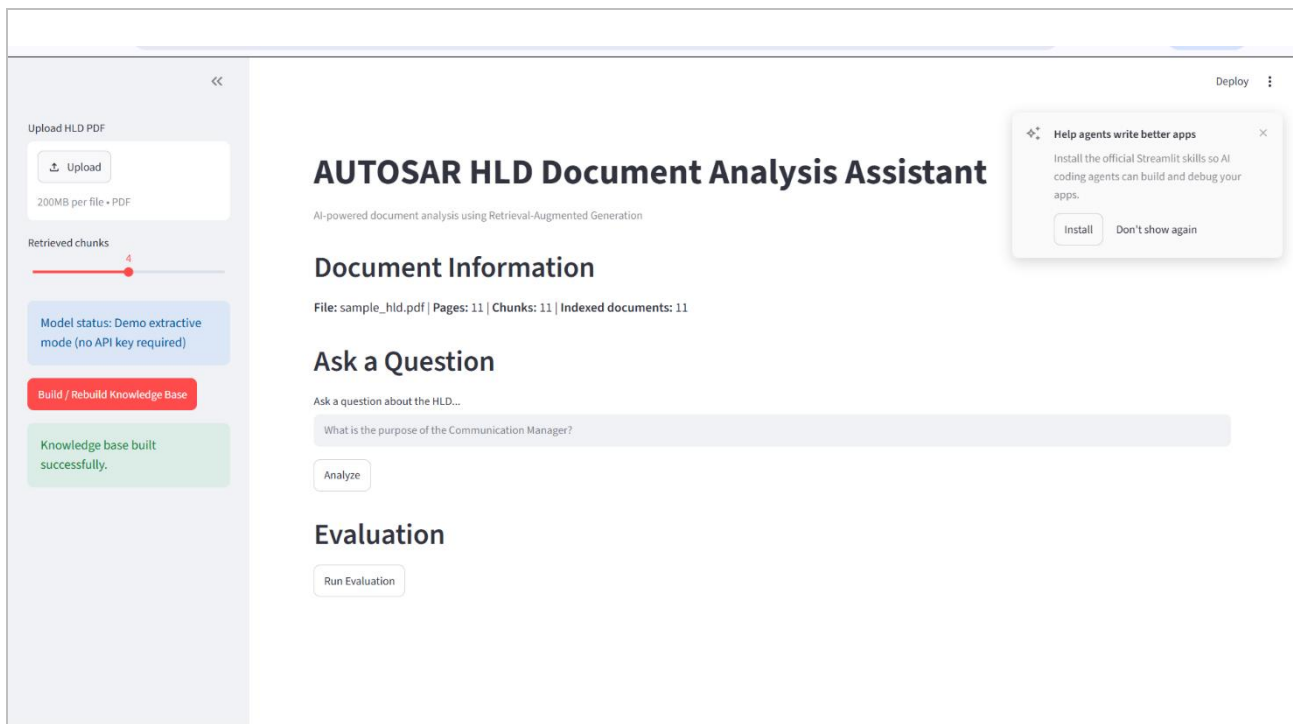


Figure 3: Knowledge base built successfully, showing 11 pages, 11 chunks and 11 indexed documents.

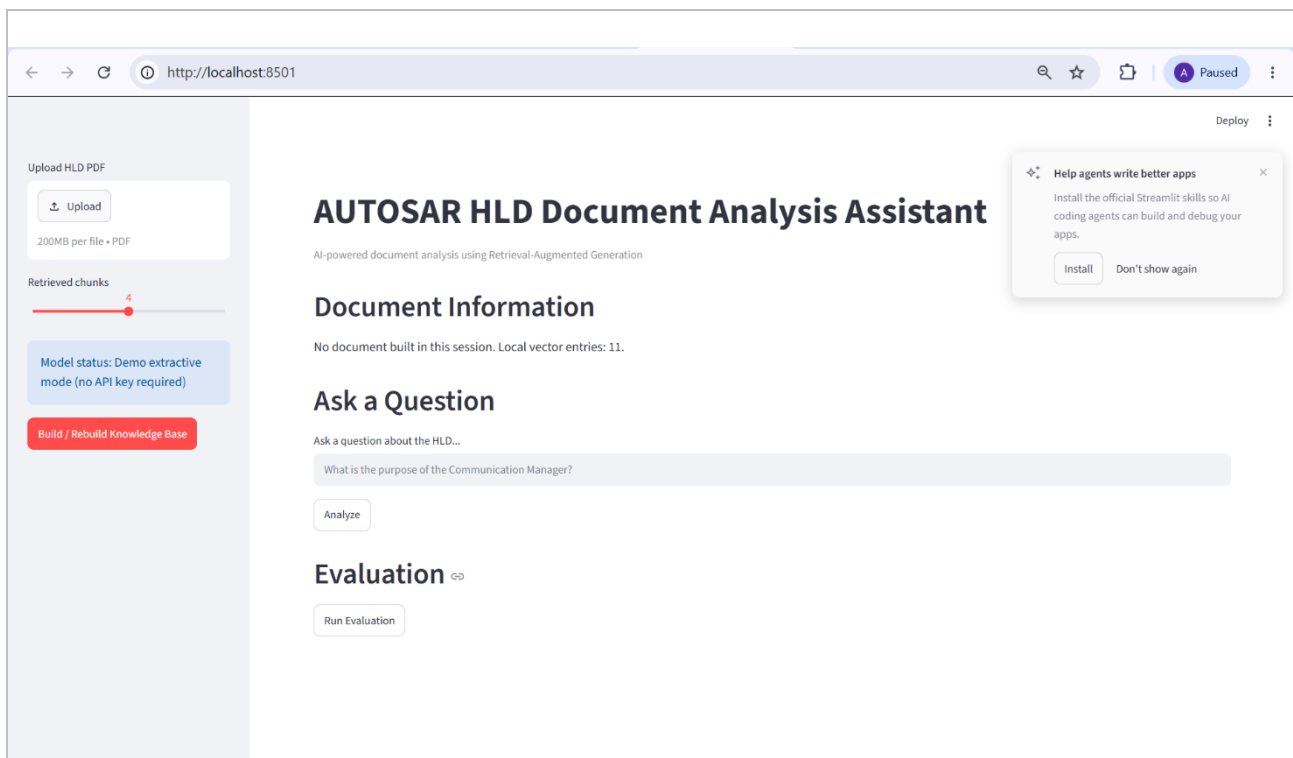


Figure 4: The question “What is the purpose of the Communication Manager?” entered in the application.

The screenshot shows the AUTOSAR HLD Document Analysis Assistant interface. On the left, there's a sidebar with 'Upload HLD PDF' (showing 'sample_hld.pdf' at 10.9KB), 'Retrieved chunks' (a progress bar at 4/11), 'Model status: Demo extractive mode (no API key required)', and a 'Build / Rebuild Knowledge Base' button. The main area has a title 'AUTOSAR HLD Document Analysis Assistant' with a subtitle 'AI-powered document analysis using Retrieval-Augmented Generation'. Below this is 'Document Information' stating 'No document built in this session. Local vector entries: 11.' The 'Ask a Question' section shows the question 'what is the purpose of the communication manager' and an 'Analyze' button. The 'AI Answer' section provides a detailed response based on retrieved HLD evidence, mentioning the Communication Architecture Communication Manager's role in coordinating communication between application components and the CAN Interface. It lists example signals like DoorLockRequest, CabinSetpoint, and DoorStateStatus. Source citations are provided at the bottom of the answer, and the response mode is set to 'demo'.

Figure 5: The answer generated for the Communication Manager question.

The screenshot shows the 'Evidence / Citations' section of the interface. It displays four numbered citations from the retrieved HLD documents, each with a brief excerpt of the relevant text. Citation 1 is from 'sample_hld.pdf - Page 6 - 5. Communication Architecture' and describes the Communication Manager's role. Citation 2 is from 'sample_hld.pdf - Page 2 - 1. Architecture Overview' and describes the Aurora platform's architecture. Citation 3 is from 'sample_hld.pdf - Page 3 - 2. Vehicle Application Manager' and describes the Vehicle Application Manager's role. Citation 4 is from 'sample_hld.pdf - Page 7 - 6. Sensor and Actuator Services' and describes the Sensor Manager's role. The sidebar on the left is identical to the previous screenshot.

Figure 6: Evidence and source citations shown with the answer, for example sample_hld.pdf, Page 6, Section 5. Communication Architecture.

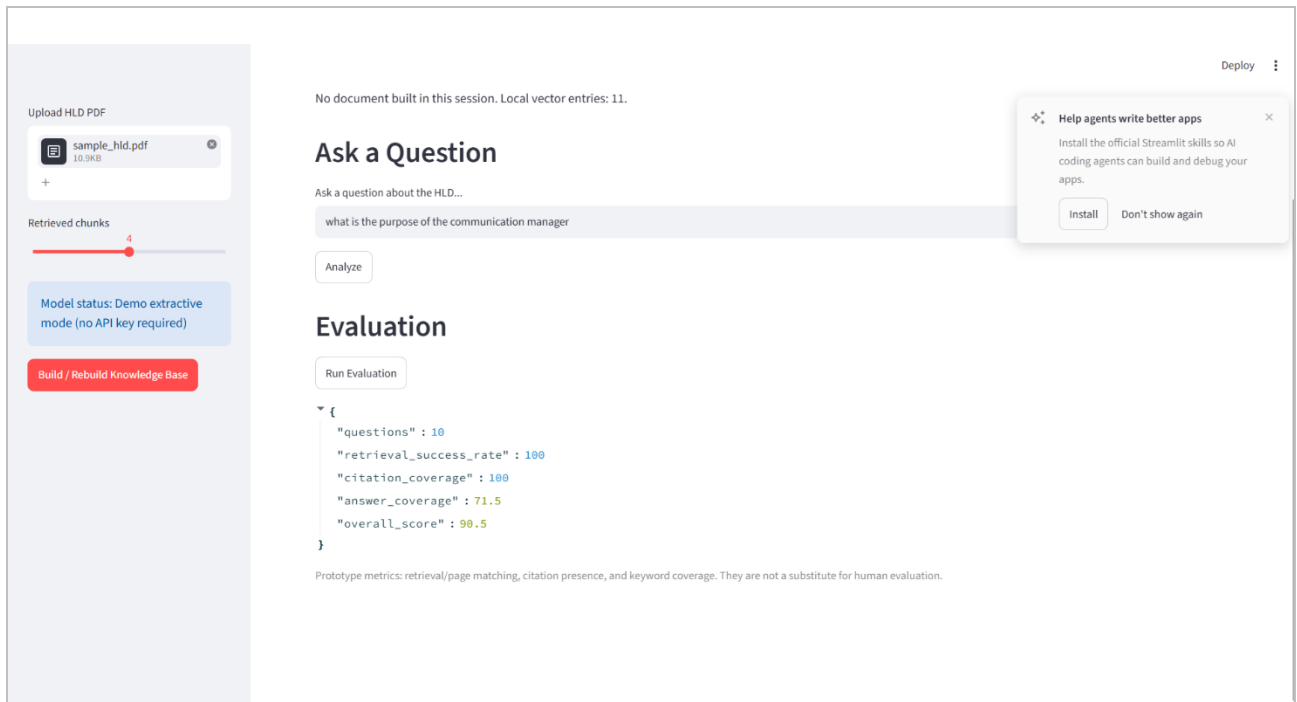


Figure 7: Evaluation results for 10 questions: 100.0% retrieval success, 100.0% citation coverage, 71.5% answer coverage and 90.5% overall score.

The screenshot shows a PowerShell terminal window with the following output:

```
File "<frozen importlib._bootstrap_external>", line 999, in exec_module
File "<frozen importlib._bootstrap>", line 488, in _call_with_frames_removed
File "C:\Users\ADITI BACHULKAR\OneDrive\Documents\Desktop\AUTOSAR_HLD_AI_Assistant\.venv\Lib\site-packages\torchvision\models\zoe
epth\image_processing_zoedepth.py", line 43, in <module>
    from torchvision.transforms.v2 import functional as tvF
ModuleNotFoundError: No module named 'torchvision'
Loading weights: 100% | 103/103 [00:00<00:00, 599.93it/s]
Loading weights: 100% | 103/103 [00:00<00:00, 1533.41it/s]
Stopping...
(.venv) PS C:\Users\ADITI BACHULKAR\OneDrive\Documents\Desktop\AUTOSAR_HLD_AI_Assistant> .venv\Scripts\activate
(.venv) PS C:\Users\ADITI BACHULKAR\OneDrive\Documents\Desktop\AUTOSAR_HLD_AI_Assistant> streamlit run Code\app.py
2026-10-07 15:45:27.545 Uvicorn server started on ::8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.46:8501

Help agents write better Streamlit apps?
Install the official Streamlit skills by running streamlit skills in your terminal.

warning: The 'fitz' API is deprecated and will be removed in future. Use 'import pymupdf' instead.
Stopping...
(.venv) PS C:\Users\ADITI BACHULKAR\OneDrive\Documents\Desktop\AUTOSAR_HLD_AI_Assistant> .venv\Scripts\activate
(.venv) PS C:\Users\ADITI BACHULKAR\OneDrive\Documents\Desktop\AUTOSAR_HLD_AI_Assistant> streamlit run Code\app.py
2026-10-07 15:46:02.898 Uvicorn server started on ::8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.46:8501

Help agents write better Streamlit apps?
Install the official Streamlit skills by running streamlit skills in your terminal.

warning: The 'fitz' API is deprecated and will be removed in future. Use 'import pymupdf' instead.
```

Figure 8: PowerShell terminal showing the Streamlit application running.

9. Reflection and Conclusions

9.1 Key learnings

- How a RAG workflow works from start to end: ingestion, retrieval and answer.

- What embeddings are and how they help to find text by meaning and not only by exact words.
- How a vector database like ChromaDB stores and searches embeddings.
- How to process a PDF document and keep page information with the text.
- The difference between an extractive answer taken from retrieved text and a fully generative LLM-based answer.
- Why citation-based answers are important: they let the user check the answer.
- That evaluating an AI system needs a proper test set and honest recording of failures, not only a few good examples.

9.2 Challenges

- Getting clean, structured information out of a PDF, especially tables.
- Choosing a chunk size that is big enough to keep the meaning but small enough to match a question well.
- Making sure the retrieval brings the correct context.
- Making sure the answer stays close to the retrieved document text.
- Judging whether an answer is correct and complete.

Environment: the final working environment uses Python 3.14.6 with the library versions listed in Section 4.2. Libraries such as sentence-transformers, torch and transformers depend on each other, so their versions have to match.

9.3 Limitations

- Only a synthetic HLD was used, so the evaluation results should not be generalized directly to real automotive HLD documents.
- The dataset is small and the evaluation set has only 10 questions.
- Text extraction from the PDF can have errors.
- The demonstrated extractive response mode may be less natural than a fully generative LLM-based response.
- This is a student demonstration and not a production automotive engineering system.
- Information available only inside complex diagrams, images or visually complicated tables may not be retrieved correctly.
- There is no similarity threshold, so a question that is not answered in the HLD may still return the closest chunks with citations.
- Answer coverage is based on keyword presence and does not measure semantic correctness.
- Latency was not measured.
- The optional LLM branch was not part of the verified evaluation.

9.4 Future improvements

- Support for more HLD formats and layouts
- Better extraction of tables and diagrams
- A larger automotive knowledge base with more documents
- A stronger evaluation with more questions and automatic scoring
- Role-based access
- Comparison between two versions of an HLD

- Improved citation handling (for example, highlighting the exact text in the PDF)
- Deployment with a local/private LLM

9.5 Final conclusion

This project built a simple retrieval-based assistant that helps a user search a synthetic AUTOSAR HLD document in natural language and see the source page and section for each answer. It helped me understand how document processing, embeddings, a vector database and answer generation can work together, and why citations matter. On the 10 predefined test questions, the prototype obtained 100.0% retrieval success, 100.0% citation coverage, 71.5% answer coverage and an overall score of 90.5%. These results apply only to the synthetic HLD and evaluation setup used here, so they do not prove performance on real automotive HLD documents. Latency was not measured. The demonstrated evaluation used the Demo Extractive Mode and not a generative LLM. The system is an assistance tool, and its answers should be checked by the user against the cited pages.

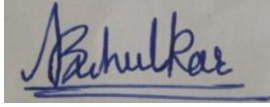
10. AI-Tool Usage Declaration

AI tools were used in two different ways: (a) as helpers while developing the project and preparing this report, and (b) as a model inside the working application. Both are listed below.

AI tool / model	Purpose	Artifact affected	Verified by Student (Y/N)
A. AI tools used for development and document preparation			
Claude (Anthropic)	Report preparation: drafting and editing this technical report.	Technical report text	[Y/N – TO BE CONFIRMED BY STUDENT]
ChatGPT / Codex (OpenAI)	Project planning, coding assistance, debugging and troubleshooting, and documentation assistance.	Project code files, design and debugging	[Y/N – TO BE CONFIRMED BY STUDENT]
B. AI models used inside the application			
Embedding model: all-MiniLM-L6-v2 (sentence-transformers 6.1.0)	Creates embeddings of chunks and questions for retrieval	Vector store content and retrieval results	[Y/N – TO BE CONFIRMED BY STUDENT]
Generative LLM (only if an LLM mode is used): not used in the verified demo evaluation. An optional OpenAI-compatible API branch exists but needs environment configuration; no model name is fixed in the code.	Answer generation in LLM mode. Not used in the demonstrated Demo Extractive Mode.	Answers shown in the app (LLM mode only)	[Y/N – TO BE CONFIRMED BY STUDENT]

11. Student Declaration

I confirm that this report represents my own understanding and contribution, that the work follows the approved case-study scope, and that I can explain every significant part of the implementation.

Signature:		Date:	7/09/2026
Name:	Aditi Ashok Bachulkar	PRN:	123B1B085